
Unifor

Apresentação T1

Alunos: Raphael Albuquerque 2413093
Vinicius Freitas 2422972
Orientador: Prof. Me. Ricardo Carubbi
Disciplina: GRAFOS
Centro de Ciências Tecnológicas - CCT
Universidade de Fortaleza - UNIFOR

Agenda

- Problema
- Modelagem
- Representação Computacional
- DFS
- Validação

PROBLEMA

- Contador de Cômodos

- Entrada :

5 8

#########

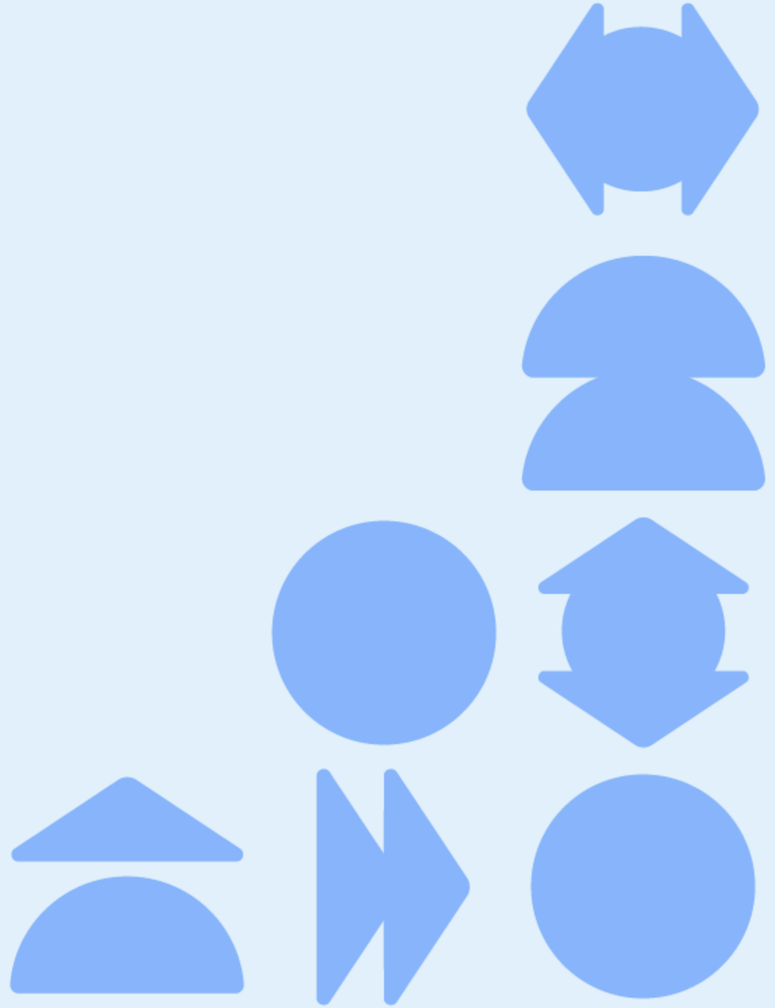
. . # . . .

. # .

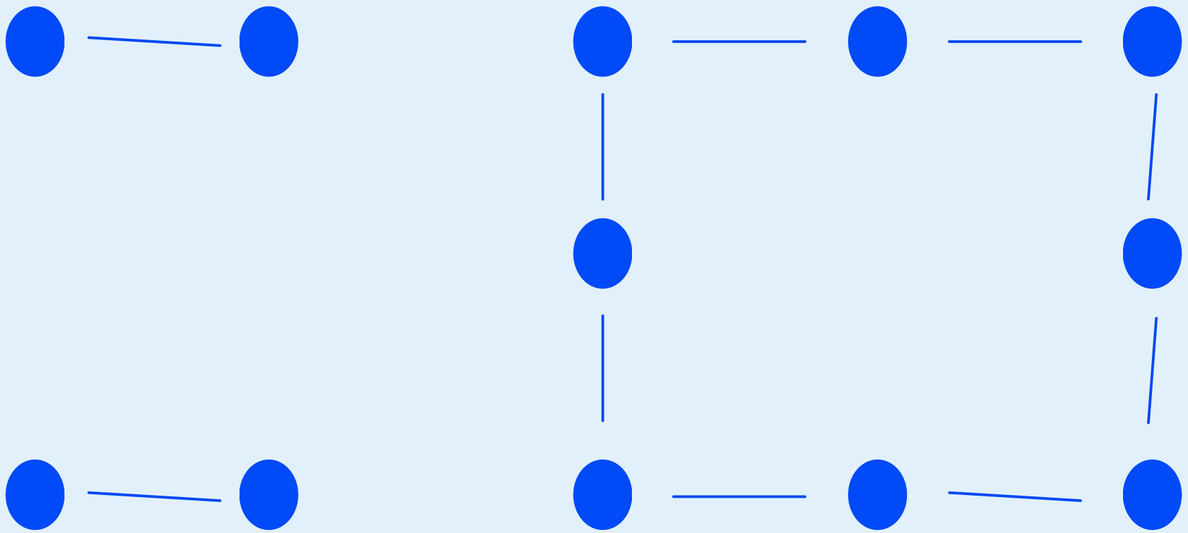
. . # . . .

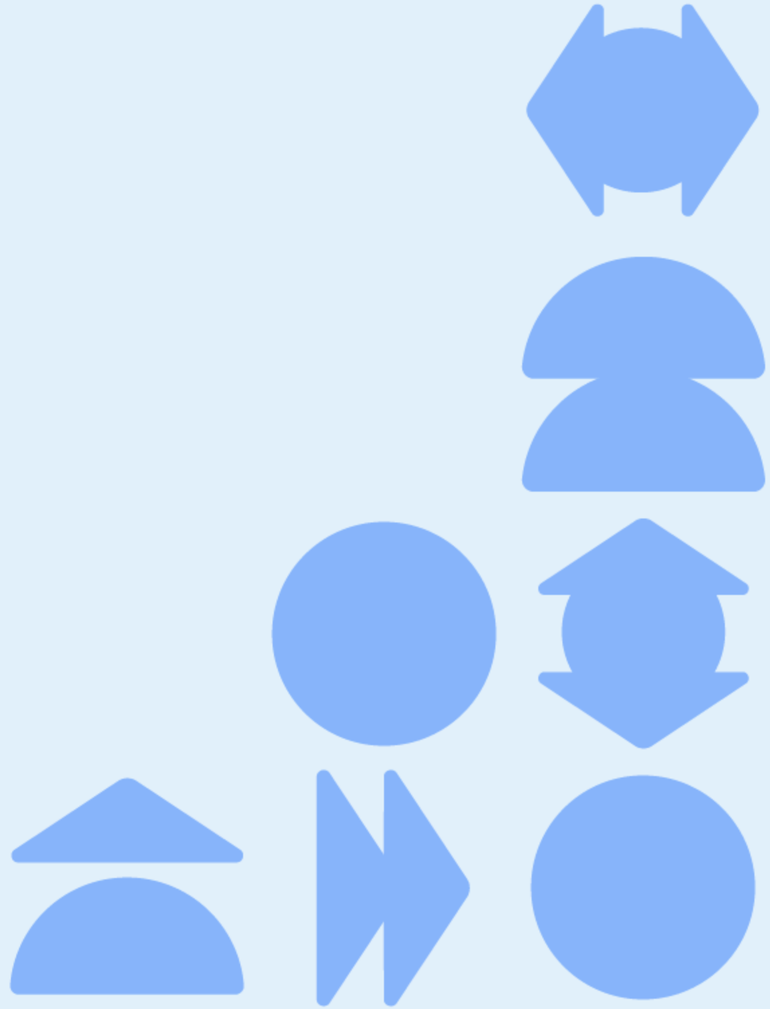
#########

Saída : 3



MODELAGEM





Código DFS

```java
public class Main {
    static int n;
    static int m;
    static char[][] mapa;
    static boolean[][] visitado;

    static void buscar(int linha, int coluna) {
        if (linha < 0 || linha >= n || coluna < 0 || coluna >= m) {
            return;
        }

        if (visitado[linha][coluna] || mapa[linha][coluna] == '#') {
            return;
        }

        visitado[linha][coluna] = true;

        buscar(linha - 1, coluna);
        buscar(linha + 1, coluna);
        buscar(linha, coluna - 1);
        buscar(linha, coluna + 1);
    }
```

```java
    public static void main(String[] args) throws Exception {

        BufferedReader leitor = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer tokenizador = new StringTokenizer(leitor.readLine());
        n = Integer.parseInt(tokenizador.nextToken());
        m = Integer.parseInt(tokenizador.nextToken());
        mapa = new char[n][m];
        visitado = new boolean[n][m];
        for (int linha = 0; linha < n; linha++) {

            mapa[linha] = leitor.readLine().toCharArray();

        }
        int quantidade = 0;
        for (int linha = 0; linha < n; linha++) {
            for (int coluna = 0; coluna < m; coluna++) {
                if (mapa[linha][coluna] == '.' && !visitado[linha][coluna]) {
                    quantidade++;
                    buscar(linha, coluna);
                }

            }
        }
        System.out.println(quantidade);

    }
}
```

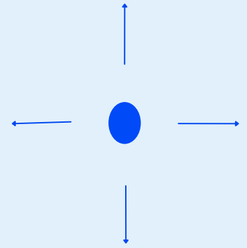
Matriz de adjacência

	0	1	2	3	4	5	6	7	8	9	A	B
0	0	1	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0	0
2	0	0	0	1	0	0	0	0	0	0	0	0
3	0	0	1	0	1	0	0	0	0	0	0	0
4	0	0	0	1	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	1	0	0	0	0	0
6	0	0	0	0	0	1	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	1	0	0	0
8	0	0	0	0	0	0	0	1	0	0	0	0
9	0	0	0	0	0	0	0	0	0	0	1	0
A	0	0	0	0	0	0	0	0	0	1	0	1
B	0	0	0	0	0	0	0	0	0	0	1	0

Lista de adjacência

(1,1) → (1,2)
(1,2) → (1,1)
(1,4) → (1,5), (2,4)
(1,5) → (1,4), (1,6)
(1,6) → (1,5), (2,6)
(2,4) → (1,4), (3,4)
(2,6) → (1,6), (3,6)
(3,1) → (3,2)
(3,2) → (3,1)
(3,4) → (2,4), (3,5)
(3,5) → (3,4), (3,6)
(3,6) → (2,6), (3,5)

Representação Implícita



```java
static void buscar(int linha, int coluna) {
    if (linha < 0 || linha >= n || coluna < 0 || coluna >= m) {
        return;
    }

    if (visitado[linha][coluna] || mapa[linha][coluna] == '#') {
        return;
    }
    visitado[linha][coluna] = true;
    buscar(linha - 1, coluna);
    buscar(linha + 1, coluna);
    buscar(linha, coluna - 1);
    buscar(linha, coluna + 1);
}
```

DFS

- Explora um caminho até o máximo possível
- Utiliza pilha ou recursão
- Percorre completamente cada componente

BFS

- Explora os vértices por níveis
- Utiliza uma fila
- Percorre primeiro os vizinhos mais próximos
- Garante menor caminho em grafos não ponderados
- Não possui o problema de estouro da pilha de recursão

Validação

Submission details

Task:	Counting Rooms
Sender:	vinifrts
Submission time:	2026-09-10 15:40:45 +0300
Language:	Java
Status:	READY
Result:	ACCEPTED

Test results

test	verdict	time	
#1	ACCEPTED	0.05 s	»
#2	ACCEPTED	0.05 s	»
#3	ACCEPTED	0.05 s	»
#4	ACCEPTED	0.05 s	»
#5	ACCEPTED	0.05 s	»
#6	ACCEPTED	0.17 s	»
#7	ACCEPTED	0.23 s	»
#8	ACCEPTED	0.24 s	»
#9	ACCEPTED	0.28 s	»
#10	ACCEPTED	0.44 s	»
#11	ACCEPTED	0.05 s	»
#12	ACCEPTED	0.06 s	»
#13	ACCEPTED	0.06 s	»
#14	ACCEPTED	0.05 s	»
#15	ACCEPTED	0.05 s	»
#16	ACCEPTED	0.05 s	»
#17	ACCEPTED	0.29 s	»
#18	ACCEPTED	0.54 s	»
#19	ACCEPTED	0.05 s	»


Unifor